

Tutorial 02: Healthcare Denials Analytics

دليل المنتور الشخصي: ازاى تبني المشروع ده من الصفر وتشرحه في أي انترفيو

(Synthetic) Claims مولدة صناعيًا على بيانات SQL Server (T-SQL) + Power BI (DAX)

ل: هيثم صالح | PL-300 | خبرة حقيقية في Denials Recovery مع Payer 70+

تنبيه أمانة: كل صف في المشروع ده داتا صناعية. الأسماء، النسب، وقصة كل Payer اتحتت في سكرت Python عن قصد. المشروع بيثبت **الطريقة** مش سلوك ال Payers الحقيقي.

1. فكرة المشروع وليه مهم (مشكلة ال RCM)
2. الداتا: Synthetic، ال Schema، ال Grain وال Keys
3. إعداد البيئة خطوة خطوة
4. شرح كل Query بالتفصيل (سكربت 00 + 01 لحد 06)
5. Power BI: الموديل، ال DAX، ال Visuals، ال Theme
6. Checklist: ازاي تعيد البناء من الصفر
7. المطبّات اللي قابلتنا (Gotchas)
8. تحضير الانترفيو: 14 سؤال بإجابات جاهزة

إزاي تذاكر الملف ده: اقرا الأقسام 1 و2 الأول عشان تفهم القصة. بعدين افتح SSMS جنب الملف ونقّذ كل Query بإيدك. القسم 8 اقراه بصوت عالي قبل أي انترفيو. كل رقم هنا اتراجع على الداتا الفعلية في SQL Server (Healthcare_Denials على \SQLEXPRESS .).

1. فكرة المشروع وليه مهم

1.1 المشكلة بلغة ال Revenue Cycle

يا هيثم، انت أصلاً عشت المشكلة دي في الشغل. بس خرينا نرتبها: العيادة بتقدم خدمة (مثلاً جلسة علاج نفسي 90837 CPT)، وتبعت Claim لل Payer (شركة التأمين). ال Payer عنده احتماليين أساسيين: يدفع أو يرفض (Denial). لما بيرفض بيبعت كود سبب اسمه **CARC** (Claim Adjustment Reason Code) وساعات كود إضافي **RARC** (Remittance Advice Remark Code) بيوضح أكثر. ال Claim المرفوض ده فلوس مش داخله لحد ما حد يشتغل عليه: يصلحه ويعيد التقديم، أو يعمل **Appeal**.

السؤال اللي كل Denials Manager بيسأله كل أسبوع:

1. مين ال Payer اللي بيرفض أكثر؟ (Denial Rate by payer)
 2. ليه بيرفض؟ بلغة ال CARC اللي فريق ال Billing يقدر يتصرف فيها.
 3. أي خدمة (CPT) بتتعرض للرفض أكثر؟
 4. هل ال Appeal بيكسب؟ (Overturn Rate) ولا بنضيّع وقت موظفين على الفاضي؟
 5. ال Claims المرفوضة بتتعد في ال A/R قد إيه؟ (Days in A/R)
 6. كام دولار معرّض للخطر دلوقتي؟ (Revenue at Risk)
- المشروع ده بيجابو الستة أسئلة دول بستة Queries (01 ل 06) وداشبورد Power BI من صفحتين.

1.2 مصطلحات RCM لازم تبقى على لسانك

المصطلح	معناه	فين في المشروع
Denial Rate	نسبة ال Claims المرفوضة من إجمالي ال Claims	Query 01 + measure Denial Rate
CARC	كود سبب التعديل/الرفض (CO-50, CO-96,...). البادئة CO = Contractual Obligation (مسؤولية المزود)، OA = Other Adjustment، PR = Patient Responsibility	عمود denial_reason
RARC	كود ملاحظة إضافي بيشرح CARC	مش موجود في المشروع، اذكره كتطوير مستقبلي
Appeal	طلب رسمي من ال Payer إنه يراجع الرفض	appeal_files , appeal_outcome
Overturn Rate	من ال Appeals اللي اتحسمت، كام % اتقلب الرفض فيها لصالحنا	Query 04 + measure Overturn Rate
Days in A/R	عدد الأيام من الخدمة لحد الدفع (أو لحد تاريخ الحساب لو لسه معلق)	Query 05
Revenue at Risk	فلوس مرفوضة ومفيش تعويض مؤكد عليها: Upheld أو Pending أو ماتعملهاش Appeal	Query 06
Timely Filing (CO-29)	ال Claim اتقدم بعد الموعد النهائي، من أصعب الرفض يتقلب	Denials (118 Claim) من ال 9.8%

ربط بخبرتك: في شغلك مع Payer 70+ شفت بنفسك إن كل Payer له "شخصية": ده بيرفض Medical Necessity كثير، ده بيرفض Prior Auth، ده Appeals عنده بتكسب. ال Generator في المشروع عمل نفس الفكرة بالكود: ست Payers بأسماء وهمية، كل واحد له Denial Rate و Appeal Rate و Overturn Rate مختلفين. عشان كده ال Queries بتطلع قصة مفهومة.

1.3 ليه ده مشروع Portfolio كويس؟

- بيثبت إنك بتفهم الدومين (CARC، Appeals، A/R) مش بس بتعرف SQL.
- بيوري Pipeline كامل: CSV ← SQL Server (BULK INSERT) ← تحليل Power BI ← T-SQL ← DAX و Theme.
- كل رقم في الداشبورديتطابق مع نتيجة SQL. ده Data Validation حقيقي.
- الأمانة: كاتب بوضوح إن الداتا Synthetic. ده بيزود مصداقيتك مش بينقصها.

قاعدة ذهبية: ماتقولش أبدًا "لقيت إن Liberty بترفض 42.5%" كأنها حقيقة عن شركة حقيقية. قول: "في الداتا الصناعية اللي ولدتها، Liberty (Payer وهمي) طلعت 42.5%، وده مقصود في ال Generator عشان أوري الطريقة".

2.1 ليه Synthetic؟

README بيشرح السبب: ملفات CMS العامة (DE-SynPUF) مفيهاش Denial Reasons على مستوى ال Claim ولا CARC ولا Appeal Outcomes. فالاختيار الأمين هو داتا صناعية متعلّمة بوضوح. وطبعًا مستحيل تستخدم داتا شغلك الحقيقية (PHI وHIPAA). ال Generator متوثّق في `data/generate_synthetic_data.py`.

2.2 ازاى اتولّدت (من السكرت الفعلي)

- `random.seed(42)` يعني لو شغلته تاني هتاخذ نفس الملف بالطبط (Reproducible).
- `Claim 4,200` (`N_CLAIMS`)، أرقامهم `CLM-2025-000001` لحد `CLM-2025-004200`.
- **Payers 6 وهميين**، لكل واحد: `share` (حصته من الحجم) و `deny_rate` و `appeal_rate` و `overturn_rate` ومدى أيام A/R للمدفوع والمرفوض وتوزيع أسباب الرفض.
- **8 كودات CPT** سلوكيات صحة نفسية (90791, 90792, 90832, 90834, 90837, 90853, 96127, 96130) بمدى سعر لكل كود ووزن حجم (90834 هو الأكثر بـ 30%).
- **6 تشخيصات** بتتوزع عشوائي (F32.1, F41.1...). موجودين في العمود بس مفيش Query بيستخدمهم.
- تواريخ الخدمة: من 06-01-2025 زائد رقم عشوائي لحد 250 يوم، فالنطاق الفعلي 06-01-2025 ل 13-09-2025.
- **"قصص" مبرمجة جوه الكود:** Liberty أعلى رافض (42% أساسي) وبتزيد 16 نقطة على 90837 وبكود CO-50 إجباري. Pinnacle بتزيد 18 نقطة على 96127 و96130 وبكود CO-96 إجباري. Apex بترفض قليل (18%) بس ال Appeals بتاعتها نادرًا تكسب (25%). Summit ال Appeals بتاعتها بتكسب (65%).
- ال Appeal: بعد ما يتقرر إن ال Claim يتعمله 15%، Appeal بيتحطوا Pending، والباقي بيتوزع بين overturned و upheld حسب `overturn_rate` الخاص بال Payer.

تنبيه مهم للانترفيو: في Query هتلاقي إن 90837 (اللي Liberty مبرمجة ترفضه) مش في أعلى القائمة (27.4%)، والأعلى 90853 (32.5%). السبب إن تأثير Liberty جزء من الحجم الكلي. وكمان لما تشوف إن كل 66 Claim مرفوض من نوع 90837 عند Liberty كان بكود (100%) CO-50، ده **مش اكتشاف**، ده سطر في الكود `code = "CO-50"`. قولها بصراحة.

2.3 Schema ال (جدول واحد: dbo.claims)

العمود	النوع	معناه
claim_id	VARCHAR(20) PK	رقم ال Claim، فريد. ده ال Primary Key
payer_name	VARCHAR(60)	اسم ال Payer (6 قيم وهمية)
cpt_code	VARCHAR(10)	كود الخدمة. نص مش رقم لأن الأكواد معزفات مش كميات
diagnosis_code	VARCHAR(10) NULL	ICD-10 (غير مستخدم في التحليل)
billed_amount	DECIMAL(10,2)	المبلغ المفوتر (مش المسموح ولا المدفوع)
claim_status	VARCHAR(10)	Paid أو Denied
denial_reason	VARCHAR(90) NULL	CARC + وصف، NULL لو مدفوع
denial_date	DATE NULL	تاريخ الرفض (بعد الخدمة بـ 12 لـ 30 يوم)
appealFiled	CHAR(1)	Y / N
appeal_outcome	VARCHAR(20) NULL	overturned / upheld / pending / not appealed
days_in_ar	INT	أيام من الخدمة للدفع (أو لتاريخ الحساب)
service_date	DATE	تاريخ تقديم الخدمة

2.4 ال Grain وال Keys

Grain: صف واحد = Claim واحد (مش Claim line، مش Patient). عدد الصفوف 4,200 = عدد ال claim_id الفريدة. **Primary Key:** claim_id. **Foreign Keys:** مفيش Foreign Keys ولا Dimension Tables: جدول مسطح (Flat). ده ببسھل التحليل بس يعني مفيش جدول Payers ولا CPT ولا Calendar. اذكر ده كـ Limitation بدل ما تخبيه.

2.5 توزيع الحالات (راجعته من الداتابيز)

claim_status	appealFiled	appeal_outcome	عدد
Paid	N	NULL	3,002
Denied	Y	overturned	309
Denied	Y	upheld	248
Denied	Y	pending	122
Denied	N	not appealed	519

المجموع: 4,200 = 1,198 + 3,002. و 1,198 = 309 + 248 + 122 + 519. احفظ الجدول ده، لأنه مفتاح فهم 04 Query و 06 Decided. Revenue at Risk = 248 + 122 + 519 = 889 ال Appeals = 309 + 248 = 557. Claims

الإجمالي المفوتر \$719,233.10، المرفوض منه \$201,183.80 (28.0% بالدولار، و 28.5% بالعدد).

3. إعداد البيئة خطوة خطوة

3.1 المطلوب

- **SQL Server Express** (instance اسمه `SQLEXPRESS` ، والسيرفر بيتكتب `SQLEXPRESS`).
- **SSMS** (أو `sqlcmd`) لتشغيل السكريبتات.
- **Python 3** لو عايز تولّد الداتا بنفسك (من غير مكتبات خارجية: `csv` و `random` و `datetime`).
- **Power BI Desktop** (Windows)

3.2 الخطوات

1. وُلّد الداتا (اختياري، الملف جاهز): من جذر المشروع شغّل `python data/generate_synthetic_data.py` . هيكيب `data/claims.csv` وهيطبع عدد الـ Claims وعدد المرفوض. (المسار نسبي، فلازم تشغله من جذر المشروع).
2. انسخ الـ **CSV** لفولدر يقدر **SQL Server** يقرأه. دي أهم نقطة. خدمة SQL Server بتشتغل بحساب خدمة (Service Account) مش بحسابك. فولدر `Downloads` محمي بصلاحيات حسابك. عشان كده السكريبت بيقرا من `C:\Users\Public\healthcare-` `denials\claims.csv` . اعمل الفولدر وانسخ الملف:

```
mkdir C:\Users\Public\healthcare-denials
copy data\claims.csv C:\Users\Public\healthcare-denials\claims.csv
```

3. شغّل `sql/00_setup_database.sql` من SSMS أو بـ `sqlcmd`:

```
sqlcmd -S .\SQLEXPRESS -C -i sql\00_setup_database.sql
```

الـ `-C` معناها "Trust Server Certificate" عشان الاتصال المشغّر يشتغل من غير شهادة موثوقة (شائع مع Express محلي).

4. اتأكد من الأرقام: آخر Query في السكريبت لازم يرجع 4200 و 1198 و 28.5.
5. شغّل **01** لـ **06** على الداتابيز `Healthcare_Denials` (`-d Healthcare_Denials` أو `USE`).
6. افتح الـ **pbix** واعمل Refresh. لو السيرفر اسمه مختلف: Transform data ثم Data source settings.

خطأ شائع: "Cannot bulk load because the file could not be opened" معناه مشكلة صلاحيات أو المسار غلط. الحل: انقل الملف لـ `C:\Users\Public` أو اديّ حساب الخدمة صلاحية Read على الفولدر.

4. شرح كل Query بالتفصيل

لكل Query هنمشي بنفس الترتيب: الكود، ليه كتبناه كده، بيعمل إيه، ليه اخترنا الحل ده، الطرق البديلة، ومثال صغير. كل النتائج اتشغلت فعلاً على الداتابيز.

4.0 سكربت setup_database.sql_00 (الإعداد والتحميل)

```
IF NOT EXISTS (SELECT 1 FROM sys.databases WHERE name = 'Healthcare_Denials')
    CREATE DATABASE Healthcare_Denials;
GO
USE Healthcare_Denials;
GO
DROP TABLE IF EXISTS dbo.claims;
CREATE TABLE dbo.claims (
    claim_id          VARCHAR(20)    NOT NULL PRIMARY KEY,
    payer_name        VARCHAR(60)    NOT NULL,
    cpt_code          VARCHAR(10)    NOT NULL,
    diagnosis_code     VARCHAR(10)    NULL,
    billed_amount      DECIMAL(10,2) NOT NULL,
    claim_status       VARCHAR(10)    NOT NULL,
    denial_reason      VARCHAR(90)    NULL,
    denial_date        DATE           NULL,
    appeal_filed       CHAR(1)        NOT NULL,
    appeal_outcome      VARCHAR(20)    NULL,
    days_in_ar         INT            NOT NULL,
    service_date       DATE           NOT NULL
);
GO
BULK INSERT dbo.claims
FROM 'C:\Users\Public\healthcare-denials\claims.csv'
WITH (FORMAT = 'CSV', FIRSTROW = 2, FIELDTERMINATOR = ',',
      ROWTERMINATOR = '0x0a', TABLOCK);
GO
UPDATE dbo.claims SET denial_reason = NULLIF(denial_reason, ''),
                    appeal_outcome = NULLIF(appeal_outcome, '');
GO
```

ليه كتبناه كده؟

- `IF NOT EXISTS ... CREATE DATABASE` و `DROP TABLE IF EXISTS` بيخلّوا السكربت **Idempotent**: تشغله 10 مرات والنتيجة واحدة (مفيش 8,400 صف بالغلط).
- `GO` مش T-SQL، ده فاصل Batch بتفهّمه SSMS و `sqlcmd`. لازم قبل `USE` عشان ال Batch اللي بعده يشتغل على الداتابيز الجديدة.
- ال Types: `DECIMAL(10,2)` للفلوس (مش `FLOAT` عشان الكسور العشرية الثنائية بتعمل أخطاء تقريب)، `DATE` للتواريخ، `CHAR(1)` لـ `Y/N`.
- ال `PRIMARY KEY` على `claim_id` بيمنع التكرار ويعمل `Clustered Index`.

بيعمل إيه؟

`BULK INSERT` هو أسرع طريق لتحميل CSV في SQL Server. `FIRSTROW = 2` يتخطى الهيدر. `FORMAT = 'CSV'` بيفقّل فهم RFC-4180 (الاقتراسات). `TABLOCK` قفل الجدول لسرعة أعلى. ال `UPDATE ... NULLIF(col, '')` بيحوّل النصوص الفاضية لـ `NULL` عشان

denial_reason IS NULL يبقى معناها حقيقي.

ملحوظة صادقة: الملف الذي بيطلّعه Python فيه نهاية سطر `\r\n` (CRLF، لأن `csv.writer` افتراضي كده)، والسكرت كاتب `ROWTERMINATOR = '0x0a'` مع `FORMAT='CSV'` التحميل نجح فعلاً وطلع 4,200 صف. بس لو غيّرت `FORMAT` أو استخدمت `Bulk` قديم، هتلاقي `\r` لاصق في آخر عمود (`service_date`). ده من أول الحاجات اللي تتأكد منها لو التحميل فشل.

الطرق البديلة

- **Import Flat File Wizard** في SSMS: أسهل بال GUI، بس مش قابل للتكرار ومش بيتحط في Git.
- **bcp** أو **OPENROWSET(BULK...)**: الأول CLI، الثاني بيقرأ من غير ما تعمل جدول.
- **Python (pandas + SQLAlchemy to_sql)**: مرّن في التنضيف، أبطأ ومحتاج `Driver`. (بالمناسبة ده مكان طبيعي لمهارة الـ `pandas` بتاعتك.)

ملخص التحقق (آخر Query في السكرت)

```
SELECT COUNT(*) AS total_claims,
       SUM(CASE WHEN claim_status = 'Denied' THEN 1 ELSE 0 END) AS denied_claims,
       CAST(100.0 * SUM(CASE WHEN claim_status = 'Denied' THEN 1 ELSE 0 END)
           / COUNT(*) AS DECIMAL(4,1)) AS denial_rate_pct
FROM dbo.claims;
```

النتيجة الفعلية: **4200 | 1198 | 28.5**. ملحوظة: `100.0 * ...` ضرورية. لو كتبت `100 * 1198 / 4200` كله Integer فالقسمة تقطع الكسور (28 مش 28.5).

Query 01: Denial Rate by Payer 4.1

```
SELECT TOP 10
       payer_name,
       COUNT(*) AS total_claims,
       SUM(CASE WHEN claim_status = 'Denied' THEN 1 ELSE 0 END) AS denied_claims,
       ROUND(100.0 * SUM(CASE WHEN claim_status = 'Denied' THEN 1 ELSE 0 END)
           / COUNT(*), 1) AS denial_rate_pct,
       ROUND(SUM(CASE WHEN claim_status = 'Denied' THEN billed_amount ELSE 0 END), 2)
       AS denied_billed_amount
FROM claims
GROUP BY payer_name
HAVING COUNT(*) >= 100
ORDER BY denial_rate_pct DESC;
```

Line by line

- `GROUP BY payer_name`: بنطوي 4,200 صف لـ 6 صفوف، واحد لكل Payer.
- `COUNT(*)`: عدد الـ Claims في كل مجموعة.
- `SUM(CASE WHEN claim_status='Denied' THEN 1 ELSE 0 END)`: ده **Conditional Aggregation**. الـ `CASE` بيحوّل كل صف لـ 1 أو 0 والـ `SUM` بيجمع. بنعد المرفوض من غير ما نعمل `WHERE` (لأن `WHERE` كانت هتشيل المدفوع من المقام).
- `100.0 * ... / COUNT(*)`: النسبة المئوية. الـ 100.0 (عشري) بتجبر القسمة تبقى `Decimal`.
- `ROUND(x, 1)`: رقم عشري واحد.
- العمود الأخير: الدولارات المرفوضة، نفس الفكرة بس بنجمع `billed_amount`.

- `HAVING COUNT(*) >= 100` : فلتّر على المجموعة بعد التجميع. ييمنع Payer صغير (3 Claims منهم 2 مرفوضين = 66%) من إنه يطلع الأول بنسبة مضللة. دي فكرة **Minimum Sample Size**.
- `TOP 10` : عندنا 6 Payers بس فمش بتقص حاجة. هي موجودة عشان لو الداتا كبرت.
- `ORDER BY denial_rate_pct DESC` SQL Server : بيسمح بال Alias في ال ORDER BY (بس مش في WHERE ولا HAVING).

ليه كتبناه كده؟ وبيعمل إيه؟

بيجاوب "مين بيرفض أكثر؟" بنسبة (مش عدد) عشان Summit عندها 1,154 Claim فطبيعي عدد الرفض عندها يبقى كبير، لكن النسبة بتقارن بشكل عادل. بنضيف الدولارات عشان النسبة لوحدها ممكن تكون Payer صغير.

denied \$	rate %	denied	total	payer
54,661.10	42.5	324	763	Liberty Care PPO
41,595.51	39.8	246	618	Pinnacle Commercial Plan
26,228.45	27.5	153	557	Gateway Health Cooperative
49,963.81	25.9	299	1154	Summit Behavioral Health Plan
20,619.41	17.7	122	688	Apex Medicare Advantage
8,115.52	12.9	54	420	Harbor State Medicaid

لاحظ إن Summit دفعت \$49,964 مرفوضة بنسبة 25.9% بس عشان حجمها كبير. نسبة ومبلغ بيحكوا قصتين مختلفتين.

مثال صغير (توضيحي، مش من الداتا): Payer عنده 5 Denied و 3 Paid. ال CASE بيطلع 1,1,0,0,0 وال SUM = 2، COUNT = 5، فالنسبة $40.0 = 2/5 * 100.0$.

ليه اخترنا الحل ده؟ والبدائل

ال Conditional Aggregation بيحسب كل المقاييس في **Scan واحد** للجدول. البديل Subquery/CTE بيحتاج مرتين:

```
-- Alternative A: CTE with two aggregates joined (works, but scans twice)
WITH tot AS (SELECT payer_name, COUNT(*) AS n FROM claims GROUP BY payer_name),
    den AS (SELECT payer_name, COUNT(*) AS d FROM claims
            WHERE claim_status = 'Denied' GROUP BY payer_name)
SELECT t.payer_name, t.n, d.d, ROUND(100.0 * d.d / t.n, 1) AS denial_rate_pct
FROM tot t JOIN den d ON d.payer_name = t.payer_name
ORDER BY denial_rate_pct DESC;

-- Alternative B: AVG trick (shortest)
SELECT payer_name,
    ROUND(100.0 * AVG(CASE WHEN claim_status = 'Denied' THEN 1.0 ELSE 0 END), 1)
FROM claims GROUP BY payer_name;
```

البديل A أطول وأبطأ ومعرّض للخطأ (لو Payer ملوش ولا Denial بيختفي بسبب INNER JOIN). البديل B قصير ومتساوي في النتيجة، لكنه أقل وضوحًا لزميل جديد. اخترنا ال CASE لأنه الأوضح والأكثر استخدامًا في الإنتاج.

Query 02: Denials by Reason 4.2

```
SELECT
    denial_reason,
    COUNT(*) AS denied_claims,
    ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 1) AS pct_of_denials,
    ROUND(SUM(billed_amount), 2) AS billed_at_risk
FROM claims
WHERE claim_status = 'Denied'
GROUP BY denial_reason
ORDER BY denied_claims DESC;
```

Line by line

- `WHERE claim_status = 'Denied'` هنا WHERE مناسبة لأننا عايزين المرفوض بس (المدفوع مالوش سبب).
- `GROUP BY denial_reason` : صف لكل CARC.
- `SUM(COUNT(*)) OVER ()` : ده سطر الدرس. `COUNT(*)` هو Aggregate عادي لكل مجموعة. `SUM(...) OVER ()` هي Window Function بدون PARTITION: بتجمع كل المجموعات مع بعض = 1,198 وبتحطها جنب كل صف. فالنسبة = عدد المجموعة / 1198.

بيعمل إيه؟ ونتيجته

بيرتب أسباب الرفض بالحجم. الفريق يعرف أي CARC يشتغل عليه الأول (Pareto):

billed \$	% of denials	claims	CARC
64,647.77	30.9	370	CO-50 not medically necessary
35,078.92	17.9	215	CO-96 non-covered charges
25,440.52	12.7	152	CO-97 bundled/already adjudicated
19,503.61	9.8	118	CO-29 timely filing
18,347.91	9.8	117	CO-16 missing information
14,823.18	7.4	89	CO-18 duplicate
7,667.74 ...	3.6, 3.4, 2.8, 1.7	43, 41, 33, 20	CO-109, PR-27, CO-197, OA-23

بالمنطق CO-16: RCM و CO-18 و CO-29 "أخطاء داخلية" يتحلوا بتحسين Front-end وال CO-50 و Billing. CO-197 محتاجين توثيق إكلينيكي و CO-96. Authorizations غالباً Benefit design. دي طريقة تحويل الرقم لأكشن، وانت عارفها من شغلك.

مثال صغير: لو 4 CO-50 و 2 Denials و 1 CO-96 و $SUM(COUNT(*)) OVER() = 4$ و CO-16. فالنسب 50% و 25% و 25%. المقام ثابت لكل الصفوف.

```
-- Alternative A: scalar subquery for the denominator
SELECT denial_reason, COUNT(*) AS n,
       ROUND(100.0 * COUNT(*) /
             (SELECT COUNT(*) FROM claims WHERE claim_status = 'Denied'), 1) AS pct
FROM claims WHERE claim_status = 'Denied'
GROUP BY denial_reason;

-- Alternative B: CROSS JOIN to a CTE total
WITH t AS (SELECT COUNT(*) AS total FROM claims WHERE claim_status = 'Denied')
SELECT c.denial_reason, COUNT(*) AS n, ROUND(100.0 * COUNT(*) / t.total, 1) AS pct
FROM claims c CROSS JOIN t
WHERE c.claim_status = 'Denied'
GROUP BY c.denial_reason, t.total;
```

الثلاثة بيدّوا نفس الجواب. ال Window أقصر وما بيكررش شرط الفلتر (لو غيّرت WHERE في مكان وفضلت الثانية، النسب تتخرب بصمت). ال Subquery أوضح للمبتدئ لكن بتكرر الشرط. ده نقاش جميل في الانترفيو.

Query 03: Denial Rate by CPT 4.3

```
SELECT TOP 10
       cpt_code,
       COUNT(*) AS total_claims,
       SUM(CASE WHEN claim_status = 'Denied' THEN 1 ELSE 0 END) AS denied_claims,
       ROUND(100.0 * SUM(CASE WHEN claim_status = 'Denied' THEN 1 ELSE 0 END)
             / COUNT(*), 1) AS denial_rate_pct,
       ROUND(SUM(CASE WHEN claim_status = 'Denied' THEN billed_amount ELSE 0 END), 2)
       AS denied_billed_amount
FROM claims
GROUP BY cpt_code
HAVING COUNT(*) >= 50
ORDER BY denial_rate_pct DESC;
```

نفس هيكل Query 01 بس بال `cpt_code` ك Grouping. الفرق الوحيد: الحد الأدنى 50 بدل 100 لأن ال CPTs أصغر حجمًا (أصغرهم 96130 بـ 218 Claim). ده Copy-and-adapt واعى، مش نسخ أعمى.

denied \$	rate %	denied	total	CPT
7,788.51	32.5	103	317	90853 group therapy
4,159.26	31.1	127	409	96127 brief assessment
25,386.88	28.8	77	267	90792
55,823.78	28.6	372	1301	90834
13,536.37	28.4	62	218	96130
11,814.47 / 37,012.20	27.4 / 27.4	109 / 196	398 / 716	90832 / 90837
45,662.33	26.5	152	574	90791

درس تحليلي: أعلى نسبة (90853) أقل كود بالدولار (7.8K\$) لأن سعره صغير. 90834 هو الأكبر بالدولار (55.8K\$) رغم إنه رقم 4 بالنسبة. ف "الألوية" = نسبة × دولار، مش نسبة بس. وده بالظبط اللي صفحة 2 في الداشبور (CPT risk matrix) بتوريه. ولاحظ إن الفروق بين الـ CPTs صغيرة (26.5 لـ 32.5) في الداتا دي، فمتبالغش في الاستنتاجات.

بديل: Window function لترتيب

```
SELECT cpt_code, denial_rate_pct,
       RANK() OVER (ORDER BY denial_rate_pct DESC) AS rnk
FROM ( SELECT cpt_code,
              100.0 * AVG(CASE WHEN claim_status = 'Denied' THEN 1.0 ELSE 0 END) AS denial_rate_pct
      FROM claims GROUP BY cpt_code HAVING COUNT(*) >= 50 ) x;
```

مفيد لو عايز الترتيب جوه النتيجة (مثلاً Top 3 لكل Payer باستخدام PARTITION BY). TOP N أبسط لما بتحتاج Top N عام.

Query 04: Appeal Overturn Rate by Payer 4.4

```
SELECT
  payer_name,
  COUNT(*) AS denied_claims,
  SUM(CASE WHEN appeal_filed = 'Y' THEN 1 ELSE 0 END) AS appeals_filed,
  ROUND(100.0 * SUM(CASE WHEN appeal_filed = 'Y' THEN 1 ELSE 0 END) / COUNT(*), 1)
    AS appeal_rate_pct,
  SUM(CASE WHEN appeal_outcome IN ('overturned', 'upheld') THEN 1 ELSE 0 END)
    AS decided_appeals,
  SUM(CASE WHEN appeal_outcome = 'overturned' THEN 1 ELSE 0 END) AS overturned,
  ROUND(100.0 * SUM(CASE WHEN appeal_outcome = 'overturned' THEN 1 ELSE 0 END)
    / NULLIF(SUM(CASE WHEN appeal_outcome IN ('overturned', 'upheld')
      THEN 1 ELSE 0 END), 0), 1) AS overturn_rate_pct
FROM claims
WHERE claim_status = 'Denied'
GROUP BY payer_name
ORDER BY overturn_rate_pct DESC;
```

Line by line

- `WHERE claim_status='Denied'` : الـ Appeals مالهاش معنى غير على المرفوض. فالمقام `denied_claims = 1,198` إجمالي.
- `appeals_filed` : كام Claim اتعمله (appeal_filed='Y').
- `appeal_rate_pct` : `appeals / denied`. "بنقاتل على كام % من الرفض؟"
- `decided_appeals` : اللي اتحسم بـ overturned أو upheld. الـ `IN (...)` بتستثني pending و not appealed.
- `overturn_rate_pct` : `decided / overturned`. ده القرار التحليلي الأهم: الـ Pending لسه ماعرفناش نتيجته، لو حطيناه في المقام نقلل النسبة ظلمًا (ده تحيز).
- `NULLIF(x, 0)` : لو `decided = 0` (Payer ما حسمش أي Appeal) القسمة هتعمل NULLIF. Divide by zero error. بترجع NULL بدل 0 فالنتيجة NULL مش خطأ. نفس فكرة `DIVIDE` في DAX.

overturn %	overturned	decided	appeal %	appeals	denied	payer
76.9	120	156	63.9	191	299	Summit
58.3	14	24	63.0	34	54	Harbor State
55.4	36	65	54.2	83	153	Gateway
50.5	54	107	49.2	121	246	Pinnacle
43.6	71	163	59.9	194	324	Liberty
33.3	14	42	45.9	56	122	Apex

مثال محسوب (Apex، أرقام حقيقية): 122 مرفوض، 42 56 Appeal، اتحسموا (overturned + 28 upheld) و14 لسه Pending. $Overt\text{urn} = 14/42 = 33.3\%$. لو حطينا ال Pending في المقام: $25\% = 14/56$ وده غلط لأن 14 منهم لسه ماعرفناش. صفوف Apex الأولى في الداتا: CLM-2025-000058 (Y, upheld, \$36.94) و CLM-2025-000106 (Y, upheld, \$139.26) و CLM-2025-000070 (N, not appealed, \$160.13).

تحذير إحصائي: Harbor State عنده 24 Appeal محسوم بس. 58.3% على 24 حالة أقل ثقة من 76.9% على 156 حالة عند Summit. في الانترفيو قول إنك كنت هتضيف Confidence Interval أو Minimum Decided Count.

القرار الأعمال

Summit بتكسب 76.9% فاعمل Appeal لكل حاجة. Apex 33.3% فكن انتقائيًا (Appeal بس لما الاحتمال عالي). Liberty بتتعمل لها Appeals كتير (59.9%) لكن بتكسب 43.6% بس، يعني جهد كتير وعائد متوسط. دي ال Appeal ROI logic.

البدائل

```
-- Alternative: pre-compute flags in a CTE, then aggregate (more readable)
WITH d AS (
  SELECT payer_name,
         CASE WHEN appeal_outcome IN ('overturned','upheld') THEN 1 ELSE 0 END AS decided,
         CASE WHEN appeal_outcome = 'overturned' THEN 1 ELSE 0 END AS won
  FROM claims WHERE claim_status = 'Denied'
)
SELECT payer_name, SUM(decided) AS decided, SUM(won) AS won,
       ROUND(100.0 * SUM(won) / NULLIF(SUM(decided), 0), 1) AS overturn_rate_pct
FROM d GROUP BY payer_name;
```

ال CTE بيمنع تكرار شرط ال CASE الطويل (في الأصل أكرر مرتين). ال trade-off: سطور أكثر. لو القياسات كتير، ال CTE بيريج العين.

Query 05: Days in A/R by Payer 4.5

```
SELECT
  payer_name,
  COUNT(*) AS total_claims,
  ROUND(AVG(CASE WHEN claim_status = 'Paid' THEN CAST(days_in_ar AS FLOAT) END), 1) AS avg_days_paid,
  ROUND(AVG(CASE WHEN claim_status = 'Denied' THEN CAST(days_in_ar AS FLOAT) END), 1) AS avg_days_denied,
  ROUND(AVG(CAST(days_in_ar AS FLOAT)), 1) AS avg_days_overall
FROM claims
GROUP BY payer_name
ORDER BY avg_days_denied DESC;
```

Line by line

- `CASE WHEN ... THEN x END` (من غير ELSE) بيرجع NULL للصفوف اللي مش مطابقة، والـ `AVG` بيتجاهل الـ NULLs. فده بيحسب متوسط المدفوع بس. لو كتبت `ELSE 0` كنت هتسحب المتوسط لتحت بالغلط!
- `AVG : CAST(days_in_ar AS FLOAT)` على INT بيرجع INT (بيقطع الكسري) في SQL Server. الـ `CAST` بيحافظ على الكسور.
- الـ `ORDER BY` على `avg_days_denied`: عشان نشوف مين أبطأ في حل المرفوض.

avg overall	avg denied	avg paid	claims	payer
53.4	86.6	28.8	763	Liberty
48.0	81.7	25.8	618	Pinnacle
38.8	77.9	24.0	557	Gateway
32.3	76.1	22.9	688	Apex
34.4	72.4	21.1	1154	Summit
23.6	64.9	17.5	420	Harbor State

الإجمالي (من Power BI): 39.0 يوم للكل، و78.9 يوم للمرفوض، والمدفوع حوالي 23.1 (حسبتها من الداتايزن). يعني المرفوض بيقتد تقريباً 3.4 ضعف. Liberty: 86.6 مقابل 28.8 = حوالي 3x.

تحفظ مهم: "avg_days_overall" بيتأثر بالـ Mix: المتوسط الكلي لأي Payer هو خليط من المدفوع والمرفوض، فـ Payer نسبة رفضه أعلى (Liberty) متوسطه الكلي بيطلع أعلى حتى لو سرعته في الدفع قريبة. وكمان `days_in_ar` في الداتا الصناعية أرقام عشوائية من مدى (الـ Generator بيحدد (12-30) للمدفع و(50-95) للمرفوض)، فالـ 3x ده تصميم مش اكتشاف. وفي الواقع DAR حقيقي بيتحسب كـ $\text{Total A/R} \div \text{Average Daily Charges}$ ، ده مقياس مختلف عن "أيام لكل Claim".

البدائل

```
-- Alternative: GROUP BY payer + status, then PIVOT (or two rows per payer)
SELECT payer_name, claim_status, ROUND(AVG(CAST(days_in_ar AS FLOAT)), 1) AS avg_days
FROM claims GROUP BY payer_name, claim_status ORDER BY payer_name, claim_status;

-- Alternative: window average, keeps every claim row (good for "vs payer average")
SELECT claim_id, payer_name, days_in_ar,
       AVG(CAST(days_in_ar AS FLOAT)) OVER (PARTITION BY payer_name) AS payer_avg
FROM claims;
```

الأول طويل (صفين لكل Payer) بس بسيط ومن. الثاني (Window) بيحافظ على مستوى الـ Claim. في تقرير الـ Payer-level، Conditional AVG بيوفر عمودين جنب بعض فيسهل المقارنة.

Query 06: Revenue at Risk 4.6

```
SELECT
    payer_name,
    COUNT(*) AS at_risk_claims,
    ROUND(SUM(billed_amount), 2) AS revenue_at_risk,
    ROUND(100.0 * SUM(billed_amount) / SUM(SUM(billed_amount)) OVER (), 1)
        AS pct_of_total_at_risk
FROM claims
WHERE claim_status = 'Denied'
    AND (appeal_outcome IN ('upheld', 'pending') OR appeal_filed = 'N')
GROUP BY payer_name
ORDER BY revenue_at_risk DESC;
```

Line by line

- **تعريف "At Risk"** (القرار الأهم): مرفوض **وغير مقلوب بال Appeal**. يعني upheld (خسرنا) أو pending (لسه مش معروف) أو appeal_filed='N' (استسلمنا). بنستخدم `appeal_filed = 'N'` بدل `appeal_outcome = 'not appealed'` عشان لو النص اتغير شوية ما نفقدش صفوف. (الأتنين بيدّوا نفس العدد هنا).
- الأقواس حول (... OR ...) إجبارية. من غيرها ال AND بتتسب قبل ال OR (أولوية المعاملات) والنتيجة بتتخرب وتدخل Paid في الحساب.
- `SUM(SUM(billed_amount)) OVER ()` : نفس نمط Query 02. ال SUM الداخلية Aggregate لل Payer والخارجية Window على كل ال Payers.

payer	at-risk claims	\$ at risk	% of total
Liberty	253	42,492.38	28.3
Pinnacle	192	31,899.31	21.2
Summit	179	30,751.94	20.5
Gateway	117	20,359.67	13.6
Apex	108	18,433.77	12.3
Harbor State	40	6,198.26	4.1

المجموع: Claim 889 و\$150,135 تقريبًا $(150,135.33 = 42,492.38 + 31,899.31 + 30,751.94 + 20,359.67 + 18,433.77 + 6,198.26)$.
وده يطابق "Revenue at Risk" measure في Power BI (\$150,135).

مثال Apex: مرفوض $122 - 14 = 108$ overturned. ده بالطبط `at_risk_claims`. وده Cross-check ذكي: `At-Risk = Denied - Overturned` (على مستوى العدد). $1,198 - 309 = 889$.

Limitation: بنستخدم `billed_amount`. في الواقع Revenue at Risk الحقيقي بيتحسب بال `Expected/Allowed Amount` (مش المفوتر) وب `probability weighting`. و Pending ممكن يتحسم لصالحنا. قولها بوضوح.

```
-- Alternative: NOT overturned instead of listing the at-risk outcomes
WHERE claim_status = 'Denied' AND ISNULL(appeal_outcome,'') <> 'overturned'

-- Alternative: percent of total via self-join / subquery
SELECT r.payer_name, r.rev,
       ROUND(100.0 * r.rev / t.total, 1) AS pct
FROM (SELECT payer_name, SUM(billed_amount) AS rev FROM claims
      WHERE claim_status='Denied' AND ISNULL(appeal_outcome,'') <> 'overturned'
      GROUP BY payer_name) r
CROSS JOIN (SELECT SUM(billed_amount) AS total FROM claims
            WHERE claim_status='Denied' AND ISNULL(appeal_outcome,'') <> 'overturned') t;
```

ال ISNULL ضروري: لو كتبت `appeal_outcome <> 'overturned'` من غير ISNULL، ال NULL مش بيساوي ولا بيخالف أي قيمة (Three-valued logic) فالصفوف دي بتختفي. هنا ال outcome مش NULL للمرفوض، بس العادة دي بتنقذك في داتا حقيقية.

Window Function مقابل Self Join (الخلاصة): لو عايز "كل صف + إجمالي المجموعة" استخدم `SUM()` `OVER()` : سطر واحد، Scan واحد. ال Self Join/CROSS JOIN بيحقق نفس النتيجة بس بكود أطول وبتتكلّم عن Join على Subquery يعني التعقيد أكبر وإمكانية تكرار الصفوف لو ال Join Key مش فريد. ال Window أنضف في كل ال Queries دي. Subquery/CTE أحسن لما تحتاج تعيد استخدام نتيجة وسيطة أو تفلتر على نتيجة Window (لأن Window ماينفعش في WHERE).

5. Power BI: الموديل وال DAX وال Visuals

5.1 الموديل (Data Model)

ال pbix فيه جدولين بس حسب reports/dax-measures.md :

- FactClaims : 4,200 صف من dbo.claims .
- _Measures : جدول فاضي بيحمل ال Measures بس (عموده الوحيد مخفي). ده Best practice: ال Measures كلها في مكان واحد بدل ما تتبعثر على الأعمدة.

ال Relationships: مفيش. جدول Fact واحد مسطح و Measures مش مربوط بحاجة. ده اختيار تبسيط، مش Star Schema. في انترفيو PL-300 هيسألوك عن Star Schema: قول بصراحة إن الموديل ده مسطح لأن الداتا مسطحة، وإنك لو توسّعت هتعمل Dim_Payer و Dim_CPT و Dim_Date وتربطهم One-to-Many Single direction ب FactClaims. كمان مفيش Date Table منفصل: عمود Service Month بيتحسب في Power Query، وده مناسب ل Trend بسيط بس مش بيدعم Time Intelligence زي SAMEPERIODLASTYEAR.

5.2 Power Query (M)

```
let
    Source = Sql.Database(".", "SQLEXPRESS", "Healthcare_Denials"),
    claims = Source{[Schema = "dbo", Item = "claims"]}[Data],
    #"Changed Types" = Table.TransformColumnTypes(claims, { ... }),
    #"Added IsDenied" = Table.AddColumn(#"Changed Types", "IsDenied",
        each if [claim_status] = "Denied" then 1 else 0, Int64.Type),
    #"Added Denial Code" = Table.AddColumn(#"Added IsDenied", "Denial Code",
        each if [denial_reason] = null then null
            else Text.BeforeDelimiter([denial_reason], " ", type text),
    #"Added Service Month" = Table.AddColumn(#"Added Denial Code", "Service Month",
        each Date.StartOfMonth([service_date]), type date)
in
    #"Added Service Month"
```

- **Navigation** مش **Native Query**: بدل ما نكتب SELECT * FROM ... بنشاور على الجدول dbo.claims. ميزتان: ال Query Folding شغال (بيتنقل الشغل للسيرفر)، ومفيش رسالة "Approve native query" المزعجة.
- **Denial Code**: Text.BeforeDelimiter(..., " ") بياخد كل اللي قبل أول مسافة، ف "...CO-50 Non-covered service" تبقى "CO-50". ده عشان تسميات الشارت تبقى قصيرة. ال if ... = null للمدفع.
- **IsDenied**: عمود 1/0. مضاف لكن ال Measures الفعلية مش بتعتمد عليه (بتفلتر على claim_status). ممكن تقول إنه helper.
- **Service Month**: أول يوم في الشهر عشان محور الزمن في ال Line chart.

5.3 Measure بالتفصيل

قبل ما نبدأ، فكرة **Filter Context**: كل Measure بتتحسب من جديد لكل خلية في ال Visual حسب الفلاتر المطبقة عليها (ال Payer على المحور، ال Slicer، ال Page filter). مثلاً COUNTROWS(FactClaims) في شارت Payer هترجع عدد صفوف ال Payer ده بس. و- CALCULATE بتعدل السياق: بتضيف أو تستبدل فلتر قبل الحساب.

Measure	DAX	بتعمل إيه + نقطة فنية
Total Claims	COUNTROWS(FactClaims)	تعد الصفوف في السياق الحالي. = 4,200 على الكل. أفضل من COUNT(claim_id) لأنها مش بتهتم بعمود.
Denied Claims	CALCULATE(COUNTROWS(FactClaims), FactClaims[claim_status]="Denied")	تضيف فلتر Denied للسياق. = 1,198. الفلتر هنا Boolean مختصر بيتحول ل FILTER على العمود، وبيكتب فوق أي فلتر ثاني على نفس العمود.
Total Billed Amount	SUM(FactClaims[billed_amount])	\$719,233.
Total Denied Amount	CALCULATE(SUM(billed_amount), claim_status="Denied")	\$201,184 (طابقت Query 01).
Denial Rate	DIVIDE([Denied Claims],[Total Claims],0)	28.5% DIVIDE يتعالج القسمة على صفر: بترجّع البديل (0) بدل خطأ. هي دايماً أفضل من "/" في ال Measures.
Denied Dollar Rate	DIVIDE([Total Denied Amount],[Total Billed Amount],0)	28.0% نسخة بالدولار. فرق بسيط عن 28.5% لأن المرفوض في المتوسط أرخص شوية.
Decided Appeals	CALCULATE(COUNTROWS(FactClaims), appeal_outcome IN {"overturned","upheld"})	557. نفس منطق 04 SQL. ال IN {...} هي مجموعة قيم.
Overturned Appeals	CALCULATE(COUNTROWS(FactClaims), appeal_outcome="overturned")	309.
Overturn Rate	DIVIDE([Overturned Appeals],[Decided Appeals])	55.5% (309/557). من غير بديل ثالث: لو مفيش Decided يرجّع BLANK مش 0. الفرق مهم: BLANK بيخفي النقطة في الشارت، أما 0 فهتبان كأن النسبة صفر وده كذب لما مفيش Appeals أصلاً.
Recovered Amount	CALCULATE(SUM(billed_amount), appeal_outcome="overturned")	\$51,049. تقريب: مفيش عمود "paid after appeal"، فبنستخدم المفوتر لل Claims المقلوبة.
Overturn Recovery Rate	DIVIDE([Recovered Amount],[Total Denied Amount],0)	25.4% من الدولارات المرفوضة استرجعت.
Average Days in AR	AVERAGE(FactClaims[days_in_ar])	39.0 على الكل.
Avg Days in AR (Denied)	CALCULATE(AVERAGE(days_in_ar), claim_status="Denied")	78.9 (راجعتها من الداتابيز: 78.897).
Revenue at Risk	CALCULATE(SUM(billed_amount), claim_status="Denied", appeal_outcome IN {"upheld","pending","not appealed"})	\$150,135. أكثر من فلتر في AND = CALCULATE بينهم. نفس تعريف 06 SQL.
At-Risk Claims	COUNTROWS	بنفس الفلاتر 889.

ليه DIVIDE وليه البديل 0 أحيانًا ومرات لا؟

Denial Rate بيرجّع 0 لو مفيش Claims (الصفر منطقي: مفيش رفض). Overturn Rate من غير بديل فيبرجّع BLANK (مفيش Appeals محسومة يبقى النسبة "مش معرّفة"). ده قرار مقصود، ودافع عنه في الانترفيو.

مثال Filter Context

```
-- In a card with slicer Payer = "Apex Medicare Advantage":
Total Claims      = 688
Denied Claims     = 122      (CALCULATE adds claim_status = "Denied" to the Apex filter)
Denial Rate       = 17.7%
Decided Appeals   = 42
Overturned        = 14
Overturn Rate     = 33.3%    (matches SQL query 04)
Revenue at Risk   = $18,434 (matches SQL query 06: 18,433.77)
```

بدائل في ال DAX

```
-- Alternative 1: use the helper column (IsDenied) instead of CALCULATE
Denied Claims v2 = SUM ( FactClaims[IsDenied] )

-- Alternative 2: variables for readability
Overturn Rate v2 =
VAR won      = CALCULATE ( COUNTROWS ( FactClaims ), FactClaims[appeal_outcome] = "overturned" )
VAR decided = CALCULATE ( COUNTROWS ( FactClaims ), FactClaims[appeal_outcome] IN { "overturned", "upheld" } )
RETURN DIVIDE ( won, decided )

-- Alternative 3: Revenue at Risk as Denied minus Recovered-style
Revenue at Risk v2 = [Total Denied Amount] - [Recovered Amount]
```

البديل 3 بيدي نفس النتيجة: $201,184 - 51,049 = 150,135$. ودي طريقة تحقق (Reconciliation) ممتازة تذكرها. و SUM(IsDenied) أسرع وأبسط، لكن CALCULATE أكثر مرونة وبيشتغل من غير عمود إضافي.

ملحوظة دقيقة: Recovered Amount مش مفلتر على Denied لكن مفيش overturned إلا في Denied أصلاً. فالنتيجة سليمة. بس لو الداتا اتغيرت، الأحوط تضيف الفلتر. وتقدر تقول إن DAX string comparison غير حساس لحالة الأحرف بينما SQL ده بيعتمد على ال Collation (الافتراضي غير حساس كمان).

5.4 الصفحات وال Visuals ولماذا

(من README وسكرين شوتس [reports/screenshots](#))

الصفحة	ال Visual	ليه اخترناه
Denials Overview :1	Cards: Billed vs Denied \$, Denial Rate, Days in AR	ال KPIs الرئيسية في أول نظرة، بالرقم الكبير.
	Bar: Denial Rate by Payer	مقارنة فئات = Bar. مرتب تنازلي ولونه بقاعدة (أحمر = تحرك دلوقتي).
	Bar: Denied \$ by CARC code (Denial Code)	أسماء قصيرة (CO-50) بفضل عمود Denial Code. ده Pareto.
	Line: Monthly trend (Service Month)	الزمن = Line. محور من Service Month.
	Slicer: Payer	يخلي الصفحة كلها تتفلتر ل Payer واحد (Filter Context يشتغل).
Appeals & Revenue :2 at Risk	Cards: \$150,135 / 889 / 55.5%	الأرقام اللي المدير عايزها.
	Table: Overturn by Payer	Table بدل Chart عشان فيه أكثر من رقم (Rate, Decided, Overturned) والقارئ بيقرن.
	Matrix: CPT risk (Denial rate >= 30% highlighted)	Conditional formatting: القاعدة بتلفت العين ل CPT اللي فوق 30%.
	Bar: Revenue at Risk by Payer	أين الفلوس؟ بالدولار.

Theme ال 5.5

فيه ملفين JSON في reports/ : denials-theme-dark.json (الداشبورد بيستخدمه، خلفية #0F172A وكروت #1E293B ولوحة teal/amber/red و denials-theme-light.json (اسمه "Healthcare Denials Light"، خلفية #F3F4F6 وألوان زي #0F766E و #B91C1C). بتستورد بـ View ثم Themes ثم Browse for themes. اللون بيحمل معنى: أحمر = اتحرك، كهرماني = راقب، أخضر مزرق = سليم. ده بيخلي القارئ يفهم من غير ما يقرأ.

6. Checklist: ازاي تعيد البناء من الصفر

1. [] ثبتت SQL Server Express + SSMS + Power BI Desktop + Python.
2. [] اكتب الـ Generator: ثابت 6 Payers، seed، بشخصيات (نسب رفض/Appeal/Overturn/أيام A/R)، 8 بمدى سعر، 10 CARC. اطبع عدد الصفوف والمرفوض.
3. [] شغله واتأكد: 4,200 صف و1,198 مرفوض (28.5%) مع 42 seed.
4. [] ضع claims.csv في C:\Users\Public\healthcare-denials\.
5. [] اكتب .setup: CREATE DATABASE / CREATE TABLE / BULK INSERT / NULLIF / Sanity check_00.
6. [] اكتب Query 01 (Conditional Aggregation + HAVING). راجع النتيجة.
7. [] 02 (Window % of total) ثم 03 (نفس 01 على CPT).
8. [] 04 (Overturn على Decided فقط + NULLIF).
9. [] 05 (AVG شرطي + CAST FLOAT) ثم 06 (تعريف Window + At Risk).
10. [] خذ Screenshot لكل نتيجة وحطها في sql/results.
11. [] Power BI: Get Data ثم SQL Server (.\SQLEXPRESS، Healthcare_Denials) ثم اختار الجدول (Navigation). أضف الأعمدة (IsDenied, Denial Code, Service Month).
12. [] أنشئ جدول Measures (Enter Data فاضي) واكتب الـ Measures.
13. [] طابق كل Measure مع (4200، 1198، 28.5%، 55.5%، \$150,135) SQL.
14. [] ابني الصفحتين، طبق الـ Theme، ضبط Conditional formatting.
15. [] وثّق: README فيه إخلاء "Synthetic"، وlimitations، وdax-measures.md.

7. المطبّات اللي قابلتنا (Gotchas)

المشكلة	السبب	الحل
BULK INSERT مش قادر يفتح الملف	SQL Server Service Account مش عنده صلاحية على Downloads	نقل ال CSV ل C:\Users\Public\healthcare-denials\
نص فاضي بدل NULL	الحقول الفاضية في ال CSV وصلت ك "	UPDATE ... NULLIF(col, '') بعد التحميل
النسب بتطلع أرقام صحيحة (28 بدل 28.5)	قسمة Integer	استخدام * 100.0 أو CAST
AVG(days_in_ar) بتقطع الكسور	AVG على INT يرجع INT	CAST(days_in_ar AS FLOAT)
Overturn في Divide by zero	Payer ممكن مفيش عنده Appeals محسومة	NULLIF(..., 0) في SQL و DIVIDE في DAX
Overturn Rate مضلل	إدخال ال Pending في المقام	المقام = Decided فقط
Payer صغير بنسبة مضللة	عينة صغيرة	HAVING COUNT(*) >= 100 (و50 ل CPT)
تركيبة AND/OR في Query 06	أولوية AND قبل OR	أقواس حول ال OR
رسالة Native Query Approval في Power BI	استخدام SQL مكتوب كنص	Navigation على الجدول (يحافظ على Folding)
أسماء أعمدة ال Spec الأصلي مختلفة	ال Spec فيه claim_amount و denial_status... والداتا الفعلية فيها billed_amount و claim_status...	اتبعنا الداتا الحقيقية (موثق في dax-measures.md)
"اكتشافات" هي في الحقيقة مبرمجة	ال Generator بيحقن القصص (Liberty CO-50) على (90837)	اذكرها بصراحة ك Synthetic design
CRLF في ال CSV	csv.writer بيكتب \r\n	'FORMAT='CSV' اشتغل؛ راقب لو غيرت الإعدادات
Denial Rate بالدولار ≠ بالعدد	28.0% مقابل 28.5%	اتفزق بين measure الاثنين (Denied Dollar Rate)

8. تحضير الانترفيو: 14 سؤال

قاعدة الشغل: كل إجابة بالإنجليزي بصيغة المتكلم، محددة، فيها أرقام من المشروع، وفيها جملة أمانة إن الداتا Synthetic. اترّب عليها بصوت عالي حتى تطلع طبيعية مش محفوظة. الإجابة الجيدة 45 ل 90 ثانية.

Q1 (Intro). Tell me about this project in two minutes.

I built an end-to-end denials analytics project: a SQL Server database, six T-SQL analysis queries, and a two-page Power BI dashboard. I spent over two years working denied claims across 70+ payers, so I wanted to build the analytics I wish I'd had. The data is fully synthetic, 4,200 claims I generated with a seeded Python script, because public CMS files don't contain denial reasons or appeal outcomes. The questions are the real ones: which payers deny most, why, do appeals pay off, and how much revenue is still at risk. For example, in my synthetic set 1,198 of 4,200 claims are denied, 28.5%, and about \$150K of \$201K denied billed dollars is still at risk after appeals. Every number in the dashboard reconciles to a SQL query. I'm clear that the patterns are built into the generator, so the project demonstrates my method and domain thinking, not real payer behavior.

الاستراتيجية: ابدأ بخبرتك الحقيقية (جملة واحدة)، بعدين ال Pipeline، بعدين رقمين، وخلص بجملة الأمانة. ماتدخلش في تفاصيل SQL إلا لو سألوا. الهدف: يفهموا إنك بتحل مشكلة business مش بتتمرن على أدوات.

Q2 (SQL). Why did you use CASE inside SUM instead of filtering with WHERE to compute denial rate?

The denominator needs all claims and the numerator only denied ones. If I put WHERE claim_status = 'Denied', the paid claims are gone and I can't compute the rate in one pass. With SUM(CASE WHEN claim_status = 'Denied' THEN 1 ELSE 0 END) divided by COUNT(*), I get both counts per payer in a single scan. For Liberty Care PPO that gives 324 denied out of 763, so 42.5%. I also multiply by 100.0, not 100, to avoid integer division, which would have shown 28 instead of 28.5 on the overall rate. An alternative is two CTEs joined together, but that scans twice and an inner join can silently drop payers with zero denials.

الاستراتيجية: وضّح الفرق بين WHERE (بيشيل صفوف قبل التجميع) و CASE (بيشتغل جوه التجميع). اذكر فخ Integer Division لأنه بيوري إنك اتحرقت منه قبل كده. ماتنساش تدي رقم حقيقي.

Q3 (SQL). Explain SUM(COUNT(*)) OVER () in your denial-reason query.

COUNT(*) is the normal aggregate per denial reason. SUM(...) OVER () is a window function with no partition, so it adds up those group counts across the whole result, which is 1,198 denied claims, and repeats that total on each row. Dividing gives each CARC's share without a second query. CO-50 had 370 claims, so 370 over 1,198 is 30.9%, and CO-96 is 17.9%. The alternative is a scalar subquery for the denominator, which works but repeats the WHERE filter, so if someone edits one filter and forgets the other, the percentages silently break. The window version keeps the logic in one place.

الاستراتيجية: اشرح "الطبقتين": Aggregate الأول بعدين Window فوقه. لو سألوا ليه مش في Window WHERE: بتتسبب بعد WHERE/GROUP BY. جابوب بهدوء وبمثال الرقم.

Q4 (SQL). How do you protect a rate calculation from divide-by-zero, and why did you exclude pending appeals?

I wrap the denominator in `NULLIF(x, 0)`, which returns `NULL` instead of throwing an error, so the payer shows a blank rather than breaking the whole query. For overturn rate, the denominator is only decided appeals, overturned plus upheld, because pending appeals have no outcome yet. Including them would understate the rate. For Apex, 14 of 42 decided appeals were overturned, 33.3%; if I had divided by all 56 filed, I'd get 25%, which is misleading since 14 of those are still open. In Power BI I used `DIVIDE` with no alternate result, so it returns `BLANK`, not zero, when nothing is decided.

الاستراتيجية: ده سؤال "هل بتفكر في الـ edge cases". قول المشكلة، الحل، والرقم. الربط بين `NULLIF` و `DIVIDE` بيوري إنك فاهم المفهوم مش بس السيانتاكس.

Q5 (SQL). Why `HAVING COUNT(*) >= 100`? What's the difference from `WHERE`?

`WHERE` filters rows before grouping, `HAVING` filters groups after aggregation. I need `HAVING` because the condition is on a group count. The point is a minimum sample size: a payer with three claims and two denials shows 67% and would top the ranking for the wrong reason. In this dataset all six payers pass easily, the smallest is Harbor State with 420 claims, so the threshold doesn't remove anything, but it's a guard I'd keep if the data grows. For CPT I used 50 because the codes are smaller; the smallest is 96130 with 218. A fair criticism is that the threshold is arbitrary. With real data I'd choose it with confidence intervals.

الاستراتيجية: اعترف إن الفلتر ما بيأثر في الداتا دي. الصراحة بتكسبك نقاط أكثر من إنك تتظاهر إنه ضروري. اذكر الـ confidence interval كتطوير.

Q6 (SQL). Walk me through how you loaded the CSV into SQL Server. What went wrong?

I used `BULK INSERT` with `FORMAT = 'CSV'`, `FIRSTROW = 2`, and `TABLOCK`, inside an idempotent script that creates the database if missing and drops and recreates the table so reruns never duplicate rows. The first problem was permissions: the SQL Server service account couldn't read my Downloads folder, so I copy the file to `C:\Users\Public\healthcare-denials`. The second: empty CSV fields arrive as empty strings in text columns, so I follow the load with an `UPDATE` using `NULLIF` to convert them to `NULL`, which makes `IS NULL` checks meaningful. Then I run a sanity query that must return 4,200 claims and 1,198 denied. One more detail: the Python csv writer emits `CRLF` while my row terminator is `0x0a`; `FORMAT = 'CSV'` handled it and the load matched, but it's the first thing I'd check if row counts were off.

الاستراتيجية: ده سؤال "عملتها بنفسك ولا لأ". التفاصيل الصغيرة (الصلاحيات، `NULLIF`، sanity check) هي اللي بتثبت. قول أخطاءك بثقة.

Q7 (DAX). Explain CALCULATE and filter context using your Revenue at Risk measure.

Every measure is evaluated per cell using the current filter context, like the payer on a chart axis plus slicers. CALCULATE modifies that context before evaluating its expression. My Revenue at Risk is CALCULATE of SUM of billed_amount, with claim_status equal to Denied and appeal_outcome in upheld, pending, not appealed. Multiple filter arguments combine with AND, and each overrides any existing filter on the same column. On a card with no slicer it returns \$150,135; with the slicer on Apex it returns about \$18,434, matching my SQL query. I tied it out: Total Denied Amount of \$201,184 minus the Recovered Amount of \$51,049 equals \$150,135.

الاستراتيجية: كمنتور يقولك: PL-300 هيسأل عن Filter Context بلا شك، فاستعمل مثال "نفس ال slicer، measure، مختلف، رقم مختلف". ال reconciliation في الآخر ببهرهم.

Q8 (DAX). Why DIVIDE instead of the / operator, and why is the alternate result different in different measures?

DIVIDE handles a zero or blank denominator safely, so visuals don't show errors or infinity. The alternate result is a business decision. Denial Rate uses 0 as the alternate because with no claims there are no denials. Overturn Rate has no alternate, so it returns BLANK when there are no decided appeals, because a 0% overturn rate would claim that every appeal lost, which isn't true, it's just undefined. BLANK also drops the point from a chart instead of plotting a misleading zero. The dashboard shows 28.5% denial rate and 55.5% overturn rate, 309 of 557 decided appeals, and both match my SQL.

الاستراتيجية: السؤال الحقيقي: هل بتفكر في معنى الرقم. شرح الفرق بين 0 و BLANK بيفرّقك عن 90% من المرشحين.

Q9 (Data modeling). Is this a star schema? What would you change?

Honestly, no. The source is one flat claim-level table, and the model has FactClaims plus a measures table with no relationships. That was a deliberate simplification for a single-table dataset, and I documented it as a limitation. With real data I'd split out Dim_Payer, Dim_CPT with descriptions and a service-line grouping, and a proper Dim_Date marked as a date table, all related one-to-many, single direction, to the fact table. I'd also add dimensions for provider and denial reason (CARC with group and RARC). Today the monthly trend uses a Service Month column from Power Query, which works for a simple line chart but doesn't support time-intelligence functions.

الاستراتيجية: متدافعش عن الموديل المسطح كأنه Star Schema. اعترف وقدم الخطة. ده بيوري نضج. و PL-300 certified يعني متوقع منك تعرف الفرق.

Q10 (Power BI). Why did you connect through navigation instead of a SQL statement in Power Query?

Navigating to dbo.claims through Sql.Database keeps query folding intact, so Power BI pushes later steps back to the server when possible, and it avoids the native query approval prompt each time someone opens the file. I then add three columns in Power Query: IsDenied as a one-or-zero helper, Denial Code using Text.BeforeDelimiter to keep just CO-50 for readable chart labels, and Service Month with Date.StartOfMonth for the trend line. If I needed heavy filtering I'd create a SQL view and navigate to that, which keeps both folding and the server-side logic.

الاستراتيجية: جملة "query folding" لوحدها بتحسّسهم إنك فاهم Power Query. جهّز مثال بديل (View) لو سألوا "طب لو محتاج SQL معقد؟".

Q11 (RCM business). Which payer would you prioritize first and what would you do?

In my synthetic data, Liberty Care PPO is the first priority: highest denial rate at 42.5%, 324 denied claims, and the largest revenue at risk at \$42,492, or 28.3% of the total \$150K. Its denials are 58.3% CO-50, medical necessity, so the fix is clinical documentation and a payer escalation rather than resubmission. Its appeals are filed on 59.9% of denials but only 43.6% are overturned, and denied claims sit 86.6 days in A/R versus 28.8 for paid. Pinnacle is second at 39.8% with 48% CO-96, which is a benefits-verification and authorization problem upstream. I'd caveat that these patterns were designed into the generator; the method is what transfers, and in a real shop I'd validate with remits.

الاستراتيجية: ماتقولش "Liberty هي الأسوأ" حقيقة. قول "في الداتا بتاعتني". اربط كل CARC بفريق مسؤول (clinical, front-end, billing) كده بتبان إنك شغلت فعلاً في ال RCM.

Q12 (RCM business). When should a team appeal, and how does your data answer that?

An appeal costs staff time, so it should be driven by expected value: overturn rate times expected recovery minus labor cost. My data gives the first piece by payer. Summit overturns 76.9% of decided appeals, 120 of 156, so appeal nearly everything. Apex overturns only 33.3%, 14 of 42, so I'd be selective and appeal only strong cases such as clear documentation or authorization on file. Overall, 309 of 557 decided appeals were overturned, 55.5%. I'd add that I excluded pending and that Harbor State's 58.3% rests on just 24 decided appeals, so I wouldn't act on it with that sample. The next step would be adding actual recovery amounts and timely-filing clocks, because CO-29 denials, 118 claims here, often can't be appealed at all.

الاستراتيجية: خبرتك الحقيقية هنا أقوى سلاح. اذكر ROI، حجم العينة، وCO-29. لو عندك قصة حقيقية قصيرة من شغلك (من غير بيانات حساسة) زوّدها بجملة واحدة.

Q13 (Behavioral). What was the hardest part of this project, and what would you do differently?

The hardest part was making the analysis honest. Because I generated the data, it was easy to produce a clean story, but I had to separate what is a finding from what I programmed. For example, all 66 of Liberty's denied 60-minute therapy sessions, CPT 90837, carry CO-50, and that is a rule in my generator, not an insight. So I documented the synthetic design, labeled limitations, and reconciled every dashboard number to SQL. Technically I also hit the BULK INSERT permissions issue and several rate-calculation traps like integer division. What I'd change: add a proper date dimension, confidence intervals on small samples, and use allowed amounts instead of billed amounts for revenue at risk.

الاستراتيجية: اختار تحدي حقيقي (الأمانة) مش "السيرفر وقع". ال HR بيحب شخص بيعترف بحدود شغله ويقول إيه التحسين. اختتم دايمًا بخطوات تالية.

Q14 (Behavioral). Why are you moving from Medicare sales and denials work into data analytics, and why should we believe you can do it?

Because I've seen the decisions that analytics should drive but often doesn't. I worked denied claims for more than two years across 70+ payers, and most of my prioritization was manual and intuition-based. I got PL-300 certified, I'm comfortable with intermediate T-SQL and Power BI, and I'm growing in pandas. This project is my proof: I took a real revenue-cycle question, built the data pipeline from CSV to SQL Server to a reconciled dashboard, and wrote down the limitations. I bring something many analysts lack, which is knowing what a CO-50 or CO-29 means operationally and who should fix it. I'm looking for a team where I can pair that domain knowledge with stronger engineering practices.

الاستراتيجية: قصة من 3 خطوات: كنت فين، اتعلمت إيه، إيه الدليل. ماتعتذر عن خلفيتك، دي نقطة قوتك. وماتبالغش في مستواك في pandas: قول "بتطور" بصراحة.

أسئلة سريعة للتدريب (ردود من جملة أو اثنين):

- CARC vs RARC ؟ (سبب الرفض مقابل ملاحظة توضيحية)
- CO vs PR vs OA ؟ (التزام المزود، مسؤولية المريض، تعديل آخر)
- ليه DECIMAL مش FLOAT للفلوس ؟ (دقة بدون أخطاء تقريب)
- الفرق بين Measure و Calculated Column ؟ (سياق الفلتر وقت العرض مقابل قيمة مخزنة وقت التحديث)
- إيه اللي هيتغير لو الداتا حقيقية ؟ (PHI/HIPAA، De-identification، Remits، Allowed amounts، مصدر موثوق)

خاتمة من المنتور: يا هيثم، قوة المشروع ده مش في تعقيد ال SQL، لأنه عمداً بسيط. قوته إنك تعرف تشرح كل سطر وكل رقم وكل قرار (ليه Decided بس؟ ليه الأقواس؟ ليه BLANK؟) وتقول بثقة "ده Synthetic". لو قدرت تعيد بناءه من الصفر من غير ما تبص في الملف، يبقى المشروع بقى بتاعك. بالتوفيق.